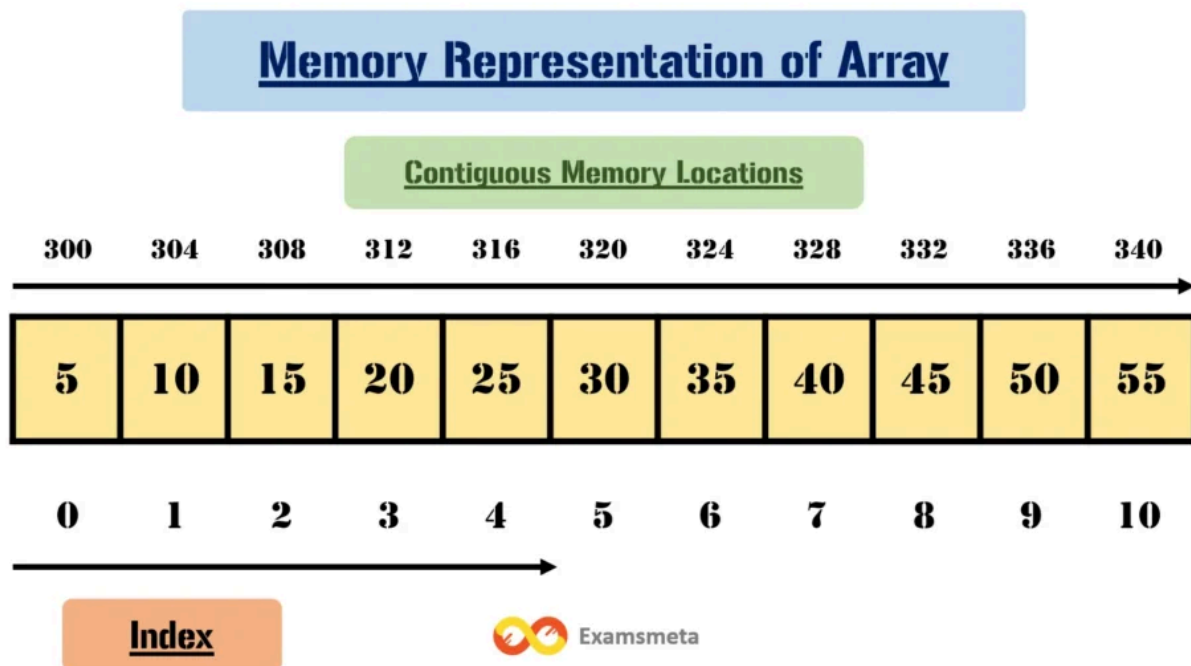


Lec 2

Unit 2 — Introduction to Arrays

Date: 14th July 2026

1. Arrays — Static Implementation



A **linear array** is a list of a finite number n of homogeneous data elements such that:

- The elements are **homogeneous** — all of the same data type.
- The elements are referenced respectively by an index (or subscript) $0, 1, 2, \dots, n-1$.
- The elements are stored in **contiguous** memory locations.

Length of the array:

$$\text{Length} = \text{Upper Bound (UB)} - \text{Lower Bound (LB)} + 1$$

- In most languages (C, C++, Java, Python), $\text{LB} = 0$, so $\text{Length} = \text{UB} + 1$.

2. Array Declaration

An array declaration needs three things:

1. **Name** of the array
2. **Data type** of the elements
3. **Number of elements** (size)

Syntax:

```
dataType arrayName[numberOfElements];
```

Example:

```
int arr[10]; // array of 10 integers
```

- Array indexing always starts from **0** (i.e., valid indices are 0 to $n-1$).

3. Memory Allocation

Type	When allocated	Size	Example
Static allocation	Compile time	Fixed, cannot change	<code>int arr[10];</code>
Dynamic allocation	Run time	Can grow/shrink as needed	<code>int* arr = new int[n];</code> (C++), <code>malloc()</code> (C)

Memory layout (conceptual):

```
Index:   0       1       2       3       4       ...      n-1
         +-----+-----+-----+-----+-----+
Value: | a0  | | a1  | | a2  | | a3  | | a4  | | ... | | an-1 |
         +-----+-----+-----+-----+-----+
Addr:  1000  1004  1008  1012  1016                (int = 4 bytes)
```

Each element's address can be computed as:

```
Address(arr[i]) = Base Address + i × size_of(dataType)
```

4. Initialization of Arrays

Syntax:

```
type arrayName[size] = {list of values};
```

Rules:

- If the number of values initialized is **less than** the declared size, the remaining elements are automatically set to **0**.

```
int arr[5] = {1, 2, 3};    // → {1, 2, 3, 0, 0}
```

- The **size may be omitted** if an initializer list is provided — the compiler infers the size from the number of elements.

```
int arr[] = {1, 8, 9};    // size automatically becomes 3
```

5. Strings in C

A **string** in C is simply an **array of characters**, terminated by a null character `'\0'`.

```
char c[] = "c string";
```

Memory diagram:

Index:	0	1	2	3	4	5	6	7	8	9
	+---+---+---+---+---+---+---+---+---+---+									
Value:	c	' '	s	t	r	i	n	g	'\0'	
	+---+---+---+---+---+---+---+---+---+---+									

- The compiler automatically appends `'\0'` at the end — this is how C knows where the string ends, since C strings don't store their length separately.

Declaration syntax:

```
char stringName[size];
```

Other ways to initialize a string:

```
char s1[] = "hello";           // implicit size (6, incl.
'\0')
char s2[6] = "hello";         // explicit size
char s3[] = {'h', 'e', 'l', 'l', 'o', '\0'}; // char-by-char with
explicit null terminator
```

6. Reading Elements of an Array

The most common way is to read one element at a time using a loop:

```
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}
```

```
arr = [int(input()) for _ in range(n)]
```

7. Properties of Arrays

- All elements are of the **same data type**.
- Elements are stored at **contiguous (continuous) memory locations**.
- Each element can be accessed directly using its index — this is called **random access**.
- Size is **fixed** once declared (in static arrays).

8. Advantages and Disadvantages of Arrays

Advantages:

- **Random access** — any element can be accessed directly in $O(1)$ time using its index.
- **Memory efficient** — no extra memory (like pointers) is needed to link elements, unlike linked lists.
- **Cache friendly** — contiguous memory storage improves performance due to spatial locality.
- **Simplicity** — easy to implement and use for storing and processing fixed-size collections.
- Forms the **base structure** for implementing other data structures (stacks, queues, hash tables, matrices).

Disadvantages:

- **Fixed size** — size must be known/declared in advance (in static arrays); cannot grow dynamically.
 - **Costly insertion/deletion** — inserting or deleting an element (except at the end) requires shifting subsequent elements, which is $O(n)$.
 - **Memory wastage** — if declared size is larger than the actual number of elements used, memory is wasted.
 - **Homogeneous only** — cannot store elements of different data types.
-

9. Operations on Arrays

Note: These four operations (traversal, insertion, deletion, searching) — along with sorting — are **not limited to arrays**. Their definitions are data-structure independent and apply equally to stacks, queues, linked lists, trees, etc. Arrays are simply the first structure we use to study them.

9.1 Traversal

Definition: Traversal means visiting each element of the data structure exactly once, typically to process or print it. This definition applies to any data structure, not just arrays — e.g., traversing a stack means visiting every element in the stack once.

Pseudocode:

```
TRAVERSE(LA, LB, UB)
Step 1: [Initialize counter] Set K = LB
Step 2: Repeat Steps 3 and 4 while K <= UB
Step 3:   Apply PROCESS to LA[K]
Step 4:   Set K = K + 1
Step 5: Exit
```

C++ implementation:

```
void traverse(int arr[], int lb, int ub) {
    for (int k = lb; k <= ub; k++) {
        cout << arr[k] << " ";    // PROCESS step
    }
}
```

Python implementation:

```
def traverse(arr, lb, ub):
    for k in range(lb, ub + 1):
        print(arr[k], end=" ")    # PROCESS step
```

Time Complexity: $O(n)$

9.2 Insertion

Definition: Insertion means adding a new element into the array — at the beginning, at the end, or at any position in between.

Since arrays are **static**, we assume there is already enough allocated space to accommodate the new element (i.e., the underlying array size N_{max} is bigger than the current logical size N).

Three cases:

a) Insertion at the beginning — all existing elements shift one position to the right.

```
Before:  [ 10 | 20 | 30 | 40 |   ]
Insert 5 at position 0
After:   [  5 | 10 | 20 | 30 | 40 ]
```

b) Insertion at the end — no shifting required; simply place the element at index N (if space is available).

```
Before:  [ 10 | 20 | 30 | 40 |   ]
Insert 50 at the end
After:   [ 10 | 20 | 30 | 40 | 50 ]
```

c) Insertion in between — all elements from the insertion point onwards shift one position to the right to make room.

```
Before:  [ 10 | 20 | 30 | 40 |   ]    Insert 25 at position K=2 (0-indexed)
Step:    shift 30, 40 → one place right
After:   [ 10 | 20 | 25 | 30 | 40 ]
```

Algorithm (general — insert at position K):

```
LA  = Linear Array
N   = number of elements currently in LA
```

K = position at which ITEM is to be inserted ($1 \leq K \leq N+1$)
ITEM = the value to be inserted

INSERT(LA, N, K, ITEM)

Step 1: [Initialize counter] Set $J = N$

Step 2: Repeat Steps 3 and 4 while $J \geq K$

Step 3: [Move Jth element downward] Set $LA[J + 1] = LA[J]$

Step 4: [Decrease counter] Set $J = J - 1$

Step 5: [End of Step 2 loop]

Step 6: [Insert element] Set $LA[K] = \text{ITEM}$

Step 7: [Reset N] Set $N = N + 1$

Step 8: Exit

Plain-English explanation of each step:

1. **Step 1:** Start a counter J at the last occupied index, N — we begin shifting from the very end of the array.
2. **Step 2:** Keep looping as long as J is at or after the insertion position K — this ensures we shift every element from the end down to (and including) position K .
3. **Step 3:** Move the element currently at position J one slot to the right (to $J+1$), freeing up its original slot and pushing existing data forward.
4. **Step 4:** Move to the previous element by decreasing J , so the next iteration shifts the element just before it.
5. **Step 5:** Once $J < K$, the loop ends — all necessary elements have been shifted, and slot K is now empty and ready.
6. **Step 6:** Place ITEM into the now-vacant position K .
7. **Step 7:** Increase N by 1 to reflect that the array now logically contains one more element.
8. **Step 8:** The algorithm is complete.

C++ implementation:

```
void insert(int arr[], int &n, int k, int item) {  
    // k is 0-indexed position; n = current number of elements  
    for (int j = n; j > k; j--) {  
        arr[j] = arr[j - 1];  
    }  
    arr[k] = item;  
    n++;  
}
```

Python implementation:

```
def insert(arr, n, k, item):
    # k is 0-indexed position; n = current number of elements
    for j in range(n, k, -1):
        arr[j] = arr[j - 1]
    arr[k] = item
    return n + 1
```

Time Complexity: $O(n)$ worst case (insertion at the beginning); $O(1)$ best case (insertion at the end).

9.3 Deletion

Definition: Deletion means removing an element from a given position and shifting subsequent elements one position to the left to close the gap.

Algorithm:

```
LA    = Linear Array
N     = number of elements currently in LA
K     = position of the element to be deleted ( $1 \leq K \leq N$ )
ITEM  = variable to hold the deleted element

DELETE(LA, N, K, ITEM)
Step 1: Set ITEM = LA[K]
Step 2: Repeat for J = K to N - 1
Step 3:   Set LA[J] = LA[J + 1]
Step 4: [End of Step 2 loop]
Step 5: Set N = N - 1
Step 6: Exit
```

Plain-English explanation of each step:

1. **Step 1:** Save the value at position `K` into `ITEM` before it gets overwritten — this is the value being removed.
2. **Step 2:** Loop through every position starting from the deleted slot `K` up to the second-last element.
3. **Step 3:** Shift each subsequent element one place to the left, overwriting the gap left by the deleted element.
4. **Step 4:** Once the loop finishes, all elements after position `K` have moved left by one, closing the gap.
5. **Step 5:** Decrease `N` by 1 since the array now logically holds one fewer element.

6. **Step 6:** The algorithm is complete; `ITEM` holds the deleted value if needed.

C++ implementation:

```
int deleteElement(int arr[], int &n, int k) {
    // k is 0-indexed position
    int item = arr[k];
    for (int j = k; j < n - 1; j++) {
        arr[j] = arr[j + 1];
    }
    n--;
    return item;
}
```

Python implementation:

```
def delete_element(arr, n, k):
    # k is 0-indexed position
    item = arr[k]
    for j in range(k, n - 1):
        arr[j] = arr[j + 1]
    return item, n - 1
```

Time Complexity: $O(n)$ worst case (deletion at the beginning); $O(1)$ best case (deletion at the end).

9.4 Searching

Definition: Searching means finding the position of a given item (`ITEM`) within the array.

a) Linear Search

Algorithm:

```
LINEAR_SEARCH(LA, N, ITEM, LOC)
Step 1: [Initialize counter] Set K = 0
Step 2: Repeat Steps 3 and 4 while K < N
Step 3:   If ITEM = LA[K], then Set LOC = K and Exit
Step 4:   Set K = K + 1
Step 5: [End of Step 2 loop]
Step 6: [Search failed] Set LOC = -1
Step 7: Exit
```

Plain-English explanation:

1. **Step 1:** Start checking from the first index, $k = 0$.
2. **Step 2:** Keep looping as long as we haven't gone past the last element.
3. **Step 3:** Compare the current element to `ITEM`; if it matches, record its position in `LOC` and stop immediately.
4. **Step 4:** Otherwise, move to the next index and try again.
5. **Step 5:** The loop naturally ends once every index has been checked without success.
6. **Step 6:** If we exit the loop without finding a match, set `LOC = -1` to signal "not found."
7. **Step 7:** The algorithm is complete.

C++:

```
int linearSearch(int arr[], int n, int item) {  
    for (int k = 0; k < n; k++) {  
        if (arr[k] == item) return k;  
    }  
    return -1;  
}
```

Python:

```
def linear_search(arr, n, item):  
    for k in range(n):  
        if arr[k] == item:  
            return k  
    return -1
```

Time Complexity: $O(n)$

b) Binary Search (array must be sorted)

Algorithm:

```
BINARY_SEARCH(LA, LB, UB, ITEM, LOC)  
Step 1: Set BEG = LB, END = UB, MID = (BEG + END) / 2  
Step 2: Repeat Steps 3 and 4 while BEG <= END and LA[MID] != ITEM  
Step 3:     If ITEM < LA[MID], then Set END = MID - 1  
           Else Set BEG = MID + 1  
Step 4:     Set MID = (BEG + END) / 2  
Step 5: [End of Step 2 loop]  
Step 6: If LA[MID] = ITEM, then Set LOC = MID
```

```
Else Set LOC = -1  
Step 7: Exit
```

Plain-English explanation:

1. **Step 1:** Set up two pointers spanning the whole array (`BEG` , `END`) and compute the middle index.
2. **Step 2:** Keep narrowing the search range as long as the range is valid and we haven't found `ITEM` yet.
3. **Step 3:** If `ITEM` is smaller than the middle element, the target must be in the left half, so move `END` before `MID` ; otherwise it's in the right half, so move `BEG` after `MID` .
4. **Step 4:** Recompute the middle index for the new, smaller range.
5. **Step 5:** The loop ends either because the item was found or the range became invalid (`BEG` `>` `END`).
6. **Step 6:** Check the final middle position — if it matches `ITEM` , record it; otherwise the item doesn't exist in the array.
7. **Step 7:** The algorithm is complete.

C++:

```
int binarySearch(int arr[], int lb, int ub, int item) {  
    int beg = lb, end = ub;  
    while (beg <= end) {  
        int mid = (beg + end) / 2;  
        if (arr[mid] == item) return mid;  
        else if (item < arr[mid]) end = mid - 1;  
        else beg = mid + 1;  
    }  
    return -1;  
}
```

Python:

```
def binary_search(arr, lb, ub, item):  
    beg, end = lb, ub  
    while beg <= end:  
        mid = (beg + end) // 2  
        if arr[mid] == item:  
            return mid  
        elif item < arr[mid]:  
            end = mid - 1  
        else:
```

```
        beg = mid + 1  
    return -1
```

Time Complexity: $O(\log n)$

9.5 Sorting

Definition: Sorting means arranging the elements of the array in a particular order (ascending or descending).

Bubble Sort (introductory sorting algorithm)

Algorithm:

```
BUBBLE_SORT(LA, N)  
Step 1: Repeat Steps 2 and 3 for K = 0 to N - 2  
Step 2:     Repeat for J = 0 to N - K - 2  
Step 3:         If LA[J] > LA[J + 1], then Interchange LA[J] and LA[J + 1]  
Step 4: [End of Step 1 loop]  
Step 5: Exit
```

Plain-English explanation:

1. **Step 1:** Run the outer pass $N-1$ times — each full pass guarantees one more element settles into its correct final position at the end.
2. **Step 2:** In each pass, scan through the unsorted portion of the array (it shrinks by one each pass, since the largest elements "bubble" to the end).
3. **Step 3:** Compare each pair of neighboring elements; if they're out of order, swap them so the larger one moves toward the end.
4. **Step 4:** After all passes complete, the array is fully sorted.
5. **Step 5:** The algorithm is complete.

C++:

```
void bubbleSort(int arr[], int n) {  
    for (int k = 0; k < n - 1; k++) {  
        for (int j = 0; j < n - k - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swap(arr[j], arr[j + 1]);  
            }  
        }  
    }  
}
```

```
}  
}
```

Python:

```
def bubble_sort(arr, n):  
    for k in range(n - 1):  
        for j in range(n - k - 1):  
            if arr[j] > arr[j + 1]:  
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
```

Time Complexity: $O(n^2)$ worst/average case; $O(n)$ best case (already sorted, with an early-exit flag optimization).

Summary Table — Operations Complexity

Operation	Best Case	Worst Case
Traversal	$O(n)$	$O(n)$
Insertion (end)	$O(1)$	$O(1)$
Insertion (beginning/middle)	$O(n)$	$O(n)$
Deletion (end)	$O(1)$	$O(1)$
Deletion (beginning/middle)	$O(n)$	$O(n)$
Linear Search	$O(1)$	$O(n)$
Binary Search (sorted array)	$O(1)$	$O(\log n)$
Bubble Sort	$O(n)$	$O(n^2)$